

Antariksh Software Stack – Development and Breakdown

Mission Profile

Max Altitude	3200m
Max Velocity	TBD
Accel Range	<18G
Recovery Type	Drogue + Main (2 Fire)
Flight Duration	TBD
Payload	Deployment post Apogee
Redundancy Requirement	Mach Barometer Spike

Hardware Interface List

What	Why	How
IMU	Acc/Gyro	I2C/SPI
Baro1	Pressure	SPI
Baro2	Pressure	SPI/I2C
LoRa	Telemetry	UART/SPI
GPS	Position	UART
SD Card	Storage	qSPI

System Requirements (what FC must do)

1. Measure
 - a. **Acceleration**
 - b. **Rotation**
 - c. **Pressure**
 - d. Battery Voltage
 - e. Position
2. Detect
 - a. Launch
 - b. Apogee
 - c. Descent
 - d. Landing
3. Determine
 - a. Altitude
 - b. Orientation
4. Do
 - a. Deploy Drogue
 - b. Deploy Main
 - c. Deploy Payload
5. Data & Communication
 - a. Telemetry data to Ground Station
 - b. Encode and Decode Packets
 - c. Log sensor data
6. Checks – Perform System check before launch
7. Redundancy
 - a. Sensor Fusion
 - b. Noise filtering
 - c. Accurate Apogee detection and fallback

Software Requirements (what code must have)

Measurement

1. Interface w IMU for Linear Accelrn and also Angular Velocity
2. Pressure measurements from Baro1 and Baro2
3. GPS data
4. Timestamp all measurements? -> Con is the overhead in terms of CPU and memory
 - a. OR -> log IMU and baro together then timestamp together
 - b. CONS of batch Timestamp: If each sensor has a different latency, (I2C,SPI,etc) and sampling rate, will cause errors in EKF
 - c. Better sample all
5. **Hardware abstraction layer** to separate application logic and sensor implementation
6. **Buffer Sensor readings** in shared/global memory structures so other modules can access it

Event Detection

1. Launch Detection should occur when the current state is ARMED and (acceleration > threshold) for a sustained time period
2. Apogee detection to occur when rate of change of altitude transitions from positive to negative for a sustained time period, this should be corroborated by the IMU and the second barometer.
3. Descent detection to occur when altitude is decreasing for a sustained time period.

Note: Sustained time period is used here to prevent noisy readings

4. Landing detection to occur only when current state is DESCENT and rate of change of altitude remains under a particular threshold for a sustained time period.
5. Software shall implement event detection only in the correct flight state (refer the flight state machine)
6. Software shall RECORD all timestamps for event detection and log it.
7. THRESHOLDS must be configurable, preferably in a dedicated config file. (//look into yaml)

State Determination

1. Estimate altitude from barometric pressure (use std atmospheric model, but have a calibration function which runs in a ARMING state)
2. Estimate Vertical velocity from rate of change of altitude

3. Orientation estimation to be done on ground using the IMU telemetry data (assuming orientation details irrelevant to flight logic, hence reducing onboard CPU load)
4. Support Sensor Fusion
5. Support Kalman Filtering

Actuation

1. Only proceed to ARMED once continuity has been verified for each pyro channel, only allow to proceed after ARMED state
2. System should only be able to control pyro channels during dedicated states (ex: DESCENT, APOGEE_DETECTED)
3. System should deploy Chutes only post apogee detection OR in case of failure.
4. Log each pyro event

Data and Communication

1. Ensure state recovery: log state info into onboard EPROM every time state changes
2. Ground station commands to be only accepted in and prior to READY state, no commands to be accepted during flight.
3. Ground station commands include : GET_SYS_HEALTH, CALIBRATE, RESET, ARM
4. Telemetry packets to include state info, timestamp, sensor data, event info(in case of a event packet)
5. Configurable telemetry rate (yaml)
6. Log all info into onboard flash and periodically flush into SD
7. Slow down telemetry rate post LANDING state

System Boot

1. Sensor Check (have buzzer and lights for people near the fc)
2. Verify Pyro channels
3. Verify battery voltage
4. Verify if onboard storage available
5. PREVENT TRANSITION to ARMED state if system check fail

Redundancy and Reliability

1. Watchdog timer for software stalls (reboot)
2. Fallback logic (in case of sensor failure)
3. Detect sensor failure
4. Apogee confirmation via secondary measure

5. Redundant apogee detection logic

RTOS Mapping		
High Priority	SensorTask	Measurement
	EstimationTask	Alti and Velocity det
Medium Priority	LoggingTask	Onboard logging
	StateMachineTask	State change logic
Low Priority	TelemetryTask	Send data to GS
	SystemHealthTask	Initialization

Note regarding RP 2350:

- What is Spinlock?
 - Spinlock is similar to shared resource lock but with a key difference: spinlock waits by repeatedly checking the lock variable rather than sleeping
 - CPU tries to acquire lock
 - if locked -> repeatedly check until unlocked
 - wastes CPU cycles but is extremely fast when the lock is held only briefly
 - Spinlock prevents the 2 cores from manipulating/accessing a shared resource at the same time, which could lead into corrupt data otherwise
 - 2040 has hardware spinlocks which are acquired automatically when you read the spinlock register
 - read spinlock register
 - > hardware automatically acquires the lock
 - if another core holds it then read returns failure
- RP 2350 does not support spinlock RP 2040
 - RP2350
 - Cortex-M33 cores only support atomic instructions and atomic instructions allow safe locking purely in software
 - Older code (2040)
 - read spinlock register
 - But on RP2350 the locking is done through SDK APIs such as
 - `spin_try_lock_unsafe()`
 - `spin_lock_unsafe_blocking()`
 - `spin_unlock_unsafe()`
- for FreeRTOS to run on a specific processor, a small hardware specific layer is required called the port and since there is no port for 2350, you have to modify the 2040 port keeping this in mind, there are a few implementations available online, refering : https://github.com/gabor-budai/FreeRTOS_PICO2350f

Software Architecture

| Data flows Upward, Control Flows Downward |

Modules

Layer	Module	Purpose
Application	Flight_State_Machine, Event_Detection, Deployment_Logig, Safety_Arming_Logic	Decides what the system should do during flight
RTOS		
Service	Telemetry, Logger, Health, Command_Parse, Timing, Config	Provides supporting system services. Service layer must never control flight decisions.
Middleware	Sensor_Fusion, State_Estimation, Filter	Processes raw sensor data into validated state estimates (altitude, velocity, orientation)
	Sensor_Processor, Time_Sync	
Driver	IMU_Driver, Baro1_Driver, Baro2_Driver, LoRa_Driver, Pyro_Drivers	Convert raw hardware interaction into usable device-level data
	GPS_Driver, Magnetometer_Driver, LED&Buzzer_Drivers, Flash_Driver, SD_Driver	
HAL		
Hardware	IMU, Baro1, Baro2, Magnetometer, GPS	Physical components that interact with the real world.
	MCU, Telemetry Module (LoRa), SD Card, Flash, RAM, SerialLink	
	PyroChannels, Battery, Buzzer, LED	

Rules

Layer	Must DO	Must NOT DO
Application	depend only on middleware + services	read raw sensor data
	independent of hardware	perform filtering or estimation
Service	provide system-wide support functions	not influence mission-critical decisions
	usable independently of flight logic	
Middleware	use only driver outputs	access hardware directly
	process data into meaningful state	make decisions
Driver	depend only on HAL (not on higher layers)	access application logic
	expose clean APIs to middleware	make decisions
HAL?	independent of specific devices (IMU, baro, etc.)	know which sensor is connected
	SPI, I2C, GPIO, ADC	interpret data
Hardware	not contain any programmable logic	process data
	only perform physical sensing and actuation	

Module Definitions

Layer	Module	Purpose		
		Description	Input	Output
Application	Flight_State_Machine	Maintains overall phase of flight. -Maintains current state -Handles State transition and enforces valid progression Does NOT detect events Only State Transitions	1. Event Flags 2. System Status (armed/unarmed)	1.Current Flight State 2.Enables/Disables actions in other modules
	Event_Detection	Detect flight events from processed data Does NOT change flight state directly	1.Altitude, Velocity, Acceleration, Position 2.Current State?	1.Event Flags
	Deployment_Logic	Decides when to deploy payload, parachute (main and drogue) Does NOT detect events Only decides when to act	1.Flight State 2.Event Flags 3.Pyro Flags 4.Config Parameters	1.Deployment Command
	Safety_Arming_Logic	controls arming/disarming prevents invalid actions before any deployment, its run through this to verify: ex: deploy_chute -> allowed?-> then go	1.Ground_Commands 2.Flight State 3.System Health	1.Arm Flag 2.Safety flag (deployment enabled?)
Service	Telemetry	Send flight data to the ground station -collect system data -build packet -encode -transmit	1.State Estimate 2.Flight State 3.Health Data	1.Telemetry Packets
	Logger	Write data to Flash then SD Buffer Data Log Events with timestamps - must not block critical tasks	1.Sensor Data 2.State Estimate 3.Flight Events	1.Log
	Health	Battery Voltage Sensor Availability Task Timing (missed deadlines)	1.Battery 2.Sensor Status 3.System Timing	1.Health Flags
	Command_Parse	receive incoming packets decode commands validate them	1.Telemetry downlink Data	1.System Commands
	Timing	maintain system clock timestamp events provide timing APIs	1. hardware timers (via HAL)	1.timestamps
	Config	Store and provide all configurable parameters load config from flash provide read access	1. stored config (flash)	1. parameters to other modules
Middleware	Sensor_Processor	raw sensor data usable unit conversion (raw counts > m/s ² , Pa) calibration (bias removal)	1. Raw sensor data	1. Processed Sensor Data
	State_Estimation	compute altitude (from pressure) compute vertical velocity compute orientation	1.fused data 2.Filtered data	1. StateEstimate: altitude velocity orientation
	Filter	low-pass filter or moving average	1. processed data	1. filtered data
	Sensor_Fusion	multiple sensors to get better estimates than any single sensor	1. filtered sensor data	1. Fused Sensor data
	Time_Sync	timestamp sensor readings synchronize IMU and barometer data provide consistent timing for estimation	1.sensor reading 2.sys clock	1. timestamped data

Layer	Module	Purpose		
Driver		Description	API	Output
	IMU_Driver	initialize IMU (register config) read accel + gyro handle SPI/I2C	imu_init() imu_read_accel() imu_read_gyro()	1.Accel x,y,z 2.Gyro x,y,z
	Baro1&2_Driver	initialize barometer read pressure + temperature	baro_read_pressure() baro_read_temperature()	1.Pressure, 2.temperature
	LoRa Driver	initialize LoRa module send packets receive packets	lora_init() lora_send(data) lora_receive(buffer)	1.tx_success 2.rx_data
	Pyro Driver	control GPIO/MOSFET for igniters perform continuity check	pyro_fire(channel) pyro_check_continuity(channel)	fire_channel(channel_id) continuity_status
	GPS_Driver	parse GPS messages (UART) extract position	gps_read_data()	latitude, longitude, altitude, time
	Magnetometer_Driver	initialize sensor read magnetic field	mag_read()	mag_x, mag_y, mag_z
	Flash_Driver	read/write flash erase sectors	flash_read(addr) flash_write(addr, data) flash_erase()	
	SD_Driver	initialize SD read/write files or blocks	sd_init() sd_write(data) sd_read(buffer)	data read/write status

Data Structures to be used

Name	Structure	Used by
SensorData	<pre>typedef struct { uint32_t timestamp; float accel_x; float accel_y; float accel_z; float gyro_x; float gyro_y; float gyro_z; float pressure_baro1; float pressure_baro2; float temperature; } SensorData;</pre>	SensorTask, Sensor_Processor, Time_Sync
ProcessedSensorData	<pre>typedef struct { uint32_t timestamp; float accel[3]; float gyro[3]; float pressure; float temperature; } ProcessedSensorData;</pre>	Sensor_Processor, Filter
FilteredSensorData	<pre>typedef struct { uint32_t timestamp; float accel[3]; float gyro[3]; float pressure; } FilteredData;</pre>	Filter, Sensor_Fusion
FusedData	<pre>typedef struct { uint32_t timestamp; float accel_corrected[3]; } FusedData;</pre>	Sensor_Fusion, State_Estimation
StateEstimate	<pre>typedef struct { uint32_t timestamp; float altitude; float velocity; }</pre>	State_Estimation, Event_Detection, Flight State Machine, Deployment Logic, Telemetry, Logger

	<pre>float orientation[3]; float acceleration[3]; } StateEstimate;</pre>	
FlightState	<pre>typedef enum { BOOT, SAFE, ARMED, BOOST, COAST, APOGEE, DESCENT, MAIN_DEPLOY, LANDING, POST_FLIGHT //tentative states } FlightState;</pre>	Flight_state_machine, deployment, telemetry, logger
Flight Event	<pre>#define EVENT_LAUNCH (1 << 0) #define EVENT_APOGEE (1 << 1) #define EVENT_DESCENT (1 << 2) #define EVENT_LANDING (1 << 3) typedef struct { uint32_t timestamp; uint16_t flags; // bitmask of events } FlightEvent;</pre>	Event detection, Flight_state_machine, deployment, telemetry, logger
DeploymentCommand	<pre>#define DROGUE (1<<0) #define MAIN (2<<0) #define PAYLOAD (3<<0) Typedef struct{ uint32_t timestamp; uint16_t flags; }DeploymentCommand;</pre>	Deployment logic, actuation task, pyro
SystemHealth	<pre>#define HEALTH_IMU_OK (1 << 0) #define HEALTH_BARO1_OK (1 << 1) #define HEALTH_BARO2_OK (1 << 2) #define HEALTH_SD_OK (1 << 3)</pre>	Health task, telemetry, flight state machine, logger

	<pre> #define HEALTH_RADIO_OK (1 << 4) typedef struct { uint32_t timestamp; float battery_voltage; uint16_t status_flags; } SystemHealth; </pre>	
TelemetryPacket	<pre> typedef struct { uint32_t timestamp; float altitude; float velocity; float gyro[3]; float accel[3]; float longitude; float latitude; uint8_t flight_state; uint16_t event_flags; uint16_t health_flags; float battery_voltage; uint16_t crc; } TelemetryPacket; </pre>	Telemetry, lora
Command	<pre> typedef enum { CMD_ARM, CMD_DISARM, CMD_START_LOGGING, CMD_STOP_LOGGING, CMD_CALIBRATE } CommandType; typedef struct { uint32_t timestamp; CommandType type; uint32_t value; // optional parameter } Command; </pre>	Command_parse, flight state machine, config
Config	<pre> typedef struct { float launch_threshold; float main_deploy_altitude; float filter_alpha; uint16_t telemetry_rate; } Config; </pre>	Config, event detection, deployment, filter, processor, telemetry

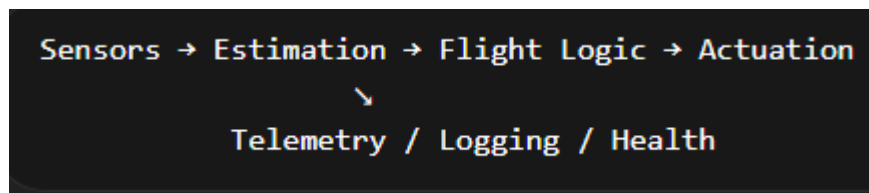


Figure 1: rtos broad overview (gpt)

RTOS Design

Task	Purpose	Freq	I/P and O/P	Timing Model
SensorTask	Read Sensordata		O/P: SensorData- > SensorQueue	Periodic
Actuation	Execute Deployment		I/P: DeploymentCommand	Event Driven
EstimationTask	SensorData processing pipeline		I/P: SensorQueue O/P: StateEstimate- > StateQueue	Periodic
FlightLogicTask	Decision Making (event detection, state machine and deployment)		I/P: StateQueue O/P: FlightEvent -> EventQueue DeploymentCommand → CommandQueue	Periodic
LoggingTask	Write logs to Flash then SD		I/P: Stateestimate and FlightEvent	Periodic
CommandTask	Incoming commands		O/P: CommandQueue	Event Driven
TelemetryTask	Sends logs to ground		I/P: StateEstimate, FlightEvent and Health	Periodic
HealthTask	Monitor System		O/P: SystemHealth	Periodic

Sequential:

[SensorTask] → SensorQueue → [EstimationTask] → StateQueue → [FlightLogicTask] → CommandQueue → [Actuation]

Parallel:

StateQueue → TelemetryTask

StateQueue → LoggingTask

EventQueue → LoggingTask

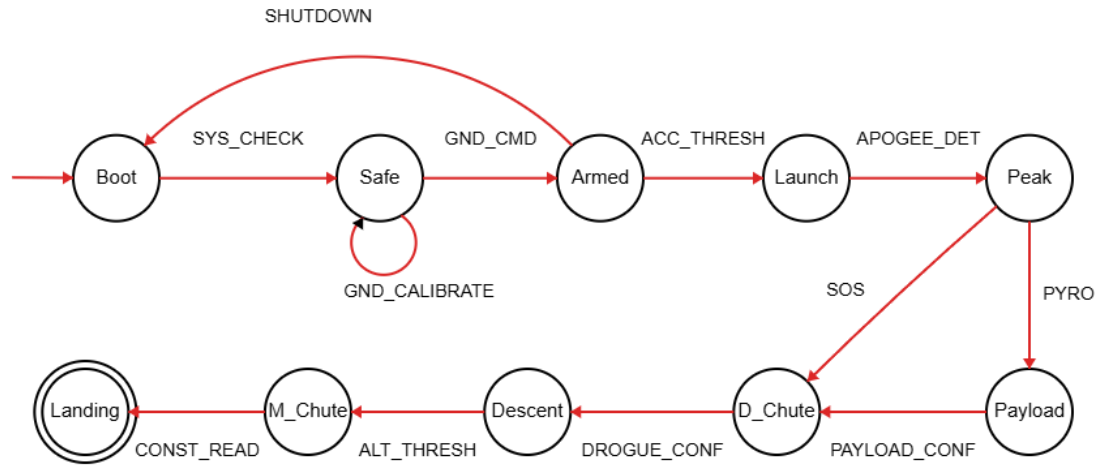
HealthTask → TelemetryTask

Dual Core

Core 0	Core 1
everything that affects flight safety	everything that affects flight safety
☐ SensorTask (1000 Hz) ☐ EstimationTask (200 Hz) ☐ FlightLogicTask (50 Hz) ☐ Actuation (pyro control)	☐ TelemetryTask ☐ LoggingTask ☐ CommandTask ☐ HealthTask

- Use queues or data buffers to communicate between cores and make it such that core 1 doesn't impact the functioning of core 0

State Machine



$M = (Q, \Sigma, \delta, q_0, F)$:

$Q = \{\text{Boot, Safe, Armed, Launch, Peak, Payload, D_chute, Descent, M_chute, Landing}\}$

$\Sigma = \{\text{sys_check, gnd_cmd, acc_thresh, apogee_det, pyro, payload_conf, sos, drogue_conf, alt_thres h, const_read, shutdown, gnd_calibrate}\}$

$q_0 = \text{Boot}$

$F = \{\text{Landing}\}$

Transition Table

Current State	Input	Next State	Transition Definition
Boot	sys_check	Safe	System health check
Safe	gnd_cmd	Armed	Arm command from ground station
Safe	gnd_calibrate	Safe	Calibrate command from ground station
Armed	acc_thresh	Launch	Vertical acceleration > threshold for some duration
Armed	shutdown	Safe	Shutdown command from ground station
Launch	apogee_det	Peak	Velocity goes from positive to negative
Peak	pyro	Payload	Firing payload pyro
Peak	sos	C_chute	Apogee detected but at less than a threshold altitude
Payload	payload_conf	C_chute	Fire drogue chute
D_chute	drogue_conf	Descent	Drogue check is positive
Descent	alt_thresh	M_chute	Altitude < threshold
M_chute	const_read	Landing	Altitude and velocity readings relatively constant

State Definitions

State	Actions
Boot	System Startup
Safe	System powered but all deployments disabled
Armed	Rocket is ready for launch and all deployment systems enabled.
Launch	Rocket has launched and is moving towards apogee
Peak	Apogee has been detected
Payload	Payload deployment has been triggered
D_chute	Drogue parachute has been deployed, expecting stabilized descent
Descent	Rocket is descending waiting to hit the altitude for main parachute
M_chute	Main parachute has been deployed, flush flash data to SD Card
Landing	Rocket has landed, go into power conservation, reduce telemetry rate.

Inter-Task Communication

The goal is to ensure:

- deterministic timing + safe data transfer + minimal coupling

Queue Definitions

Queue	Producer	Consumer Task(s)	Data Type	Frequency	Purpose
SensorQueue	SensorTask	EstimationTask	SensorData	TBD	Raw sensor transfer
StateQueue	EstimationTask	FlightLogic, Telemetry, Logging	StateEstimate	TBD	Estimated state
EventQueue	FlightLogicTask	Telemetry, Logging	FlightEvent	Event-based	Flight events
DeploymentQueue	FlightLogicTask	ActuationTask	DeploymentCommand	Event-based	Deployment commands
TelemetryQueue	TelemetryTask	LoRa Driver	TelemetryPacket	TBD	Radio transmission
LogQueue	Multiple	LoggingTask	LogRecord	TBD	Data logging
CommandQueue	CommandTask	FlightLogicTask	Command	Event-based	Ground commands

Event Signalling

Event Flag	Set By	Used By	Purpose
EVENT_APOGEE_DETECTED	FlightLogicTask	ActuationTask	Trigger deployment sequence
EVENT_PAYLOAD_DEPLOYED	ActuationTask	Logging, Telemetry	Confirm payload deployment
EVENT_DROGUE_DEPLOYED	ActuationTask	Logging, Telemetry	Drogue chute confirmation
EVENT_MAIN_DEPLOYED	ActuationTask	Logging, Telemetry	Main chute confirmation
EVENT_LANDING_DETECTED	FlightLogicTask	All tasks	End-of-flight handling

Some events are better handled using event groups instead of queues. These allow tasks to wait for specific events without polling.

Example flow:

FlightLogicTask-> EventGroupSetBits(APOGEE_EVENT)-> ActuationTask-> wait for APOGEE_EVENT

Shared Resource Protection

Certain hardware must be protected with mutexes.

Resource	Protected By Mutex	Accessed By
SD Card	Yes	LoggingTask
Flash Memory	Yes	LoggingTask
Radio Module	Yes	TelemetryTask
I2C/SPI Bus	Yes (optional)	SensorTask

Design Rules

Follow these rules strictly:

1. No shared global state between tasks
2. All data must move through queues
3. High-priority tasks must never block on slow tasks
4. Logging and telemetry must never delay flight logic
5. Queue sizes must be bounded

Interfaces (API's)

Application Layer

Module

Flight State Machine

APIs

```
flight_sm_init()
flight_sm_get_state()
flight_sm_update(const FlightEvent*,
FlightState*)
```

Event Detection

```
event_detector_init()
detect_events(const StateEstimate*,
FlightEvent*)
```

Deployment Logic

```
deployment_logic_init()
deployment_logic_update(FlightState, const
StateEstimate*, DeploymentCommand*)
```

Safety / Arming Logic

```
safety_can_arm(const SystemHealth*)
safety_can_deploy(FlightState, const
DeploymentCommand*)
```

Service Layer

Module

Telemetry

APIs

```
telemetry_init()
telemetry_build_packet(...)
telemetry_send(...)
```

Logger

```
logger_init()
logger_log_state(...)
logger_log_event(...)
```

Health Monitoring

```
health_init()
health_update(SystemHealth*)
```

Command Parser

```
command_parser_init()
command_parse(uint8_t*, Command*)
```

Config Service

```
config_load(Config*)
config_save(const Config*)
config_get()
```

Timing Service

```
timing_now_ms()
timing_now_us()
```

Middleware Layer

Module

APIs

Sensor Processor	sensor_processor_init() sensor_processor_process(...)
Filter	filter_init() filter_apply(...)
Sensor Fusion	sensor_fusion_init() sensor_fusion_update(...)
State Estimation	state_estimator_init() state_estimator_update(...)
Time Sync	time_sync_init() time_sync_stamp(...)

Driver Layer

Module	APIs
IMU Driver	imu_init() imu_read_accel(...) imu_read_gyro(...) imu_read_all(...)
Barometer Drivers	baro1_init() baro2_init() baro1_read_pressure(...) baro2_read_pressure(...)
Magnetometer Driver	mag_init() mag_read(...)
GPS Driver	gps_init() gps_read(...)
LoRa Driver	lora_init() lora_send(...) lora_receive(...)
Pyro Driver	pyro_init() pyro_fire(...) pyro_check_continuity(...)
LED / Buzzer Drivers	led_set(...) buzzer_beep(...)
Flash Driver	flash_init() flash_read(...) flash_write(...) flash_erase(...)

SD Driver

sd_init()
sd_write(...)
sd_read(...)

RTOS Tasks

Module

Tasks

APIs

sensor_task(...)
estimation_task(...)
flight_logic_task(...)
actuation_task(...)
telemetry_task(...)
logging_task(...)
health_task(...)
command_task(...)

System

Module

System Control

APIs

system_init()
system_start()

Config

TBD

Error Handling Strategy

TBD

Logging and Telemetry Format

TBD